

Executive Summary

Picture a farmer's scribbled notebook – chaos on paper – and imagine AI taming it. This report dives deep into turning those messy, multilingual farm ledgers into neat JSON. We found state-of-the-art research and tools across handwriting OCR (TrOCR, Vision Transformers 【7†L19-L27】), OCR-free document understanding (e.g. Donut 【9†L24-L32】), and advanced vision-language models (like PaddleOCR-VL 【47†L28-L37】 and Qwen2.5-VL 【36†L28-L36】) that can parse complex forms end-to-end. Open-source OCR toolkits (PaddleOCR, EasyOCR, DocTR, Kraken) offer offline engines with multi-language/handwriting support. Leading SaaS products (Google Doc AI, Azure, AWS Textract, Rossum, Nanonets, ABBYY, etc.) provide powerful APIs – but often trade off flexibility and cost. For image-to-JSON tasks, modern VLMs (MiniCPM-V, Florence-2, Claude/Anthropic's Claude Opus, Google Gemini) show promise in reasoning on visual docs and structured output.

The recommended pipeline combines image enhancement, layout detection (e.g. table/line detectors), specialized HTR (possibly a ViT-based model or PaddleOCR), and a document understanding component (like a transformer-based parser) to extract fields and segment transactions. We integrate domain logic for farm categories and keep a human-in-the-loop for low-confidence cases. The result: high-accuracy farm bookkeeping with auditable JSON output and a practical roadmap to production.

Key risks include handwriting variability, mixed languages, and the brittleness of current OCR models on messy input. We suggest innovation in “Farm Ledger AI” – e.g. fine-tuned models for Hindi/Marathi crop terms or end-to-end “farmbook” VLMs. In short, by blending proven document-AI techniques with a dash of Gen Z skepticism (“No cap, farm OCR is hard but doable”), GramIQ can revolutionize rural finance data capture.

Literature Survey

Handwritten OCR & HTR

- **Handwritten Text Recognition: A Survey (2025)** – Garrido-Muñoz *et al.* (arXiv)

Broad survey covering CNN/RNN/Transformer approaches, datasets, and hybrid OCR systems. It highlights that modern HTR often uses encoder-decoder or CTC models (CNN+LSTM/Transformer) and large datasets. As a comprehensive reference, it confirms the need for robust HTR for varied handwriting but has no single new model. *Relevance:* Sets context on HTR challenges (scribbles, languages). *Weakness:* Survey only, not a model.

- **TrOCR: Transformer-based OCR (2021)** – Li *et al.* (arXiv) 【7†L19-L27】

Abstract: An end-to-end OCR using a pre-trained Vision Transformer encoder plus a text Transformer decoder. TrOCR is simple and data-efficient: it's pre-trained on synthetic data and fine-tuned on labeled text. *Contribution:* First to leverage combined ViT+text Transformer for OCR, excelling on printed, handwritten, and scene text recognition. *Architecture:* Vision Transformer (ViT) encoding the image + BART/GPT-style text decoder generating wordpieces. *Relevance:* Demonstrates pure-Transformer HTR achieves SOTA. Could be adapted for cursive farm notes. *Weakness:* Heavy model,

needs GPU; performance depends on pre-training; may struggle with extremely messy text. *Prod-ready*: Microsoft open-sourced code and a HuggingFace model, so it's deployable (though large). *Citation*: TrOCR "outperforms current state-of-art models on printed, handwritten and scene text recognition" [7†L19-L27] .

- **HTR with Vision Transformer (2024)** – Zhang *et al.* (arXiv)

Introduces **HTR-VT**, a vision-only Transformer for handwriting. It uses minimal modifications to a standard ViT and achieves strong accuracy on IAM and Bentham handwriting datasets without pretraining. *Relevance*: Specialized on handwriting; data-efficient, showing pure-ViT is viable. *Weakness*: Novelty unclear beyond TrOCR concept; may need lots of data.

OCR-free & Document Understanding

- **Donut: OCR-free Document Understanding (2022)** – Kwak *et al.* (ECCV) [9†L24-L32]

Abstract: Current document AI pipelines split OCR and understanding, causing error propagation. Donut is the first *end-to-end* model that reads a document image and outputs structured text (like JSON) without intermediate OCR. It uses a single Transformer (Encoder-Decoder) pre-trained on synthetic data (SynthDoG). *Contribution*: OCR-free parsing of forms/invoices, achieving SOTA on multiple benchmarks with one model. *Architecture*: Vision Transformer as encoder (process image patches) + text Transformer decoder to generate output sequence (JSON). *Relevance*: Exactly fits "image-to-JSON" pipeline, removing OCR brittleness [9†L24-L32] . Could handle free-form notes. *Weakness*: Needs huge pretraining; may hallucinate if domain mismatch. Not trivial to fine-tune for farm domain. *Prod-ready*: Code & weights available, licensed MIT, but heavy compute.

- **Pix2Struct (2023)** – Gan *et al.* (ICML) [34†L17-L27]

Google's Pix2Struct is a large image-to-text model pre-trained on screenshots with masked region parsing (to simplified HTML). It's then fine-tuned on visual tasks. *Contribution*: Unified approach: pretrain on UI screenshots, fine-tune on vision+text tasks. Achieves SOTA on 6/9 cross-domain vision-LM tasks. *Architecture*: ViT-based encoder-decoder (up to 3B params). *Relevance*: It's OCR-free too and could be adapted to ledger images to output structured data. *Weakness*: Massive (billions of params), requires TPU-level resources.

- **DocFormer (2020)** – Appalaraju *et al.* (AAAI) [23†L24-L34]

A multi-modal Transformer for documents: it fuses text (from OCR), image patches, and 2D spatial embeddings via cross-attention. Pretrained with masked language/image tasks. Beating baselines on various form/receipt datasets. *Relevance*: Shows benefit of combining vision/text/spatial for KIE. *Weakness*: still requires an OCR step.

- **LayoutLMv2 (2020)** – Xu *et al.* (ACL) [12†L25-L33]

Extends BERT to documents by embedding text+position+image. Uses masked visio-linguistic pretraining tasks. Achieves 84.2% F1 on FUNSD form understanding (vs 79.95% without layout) [12†L28-L33] . *Relevance*: Strong baseline for key info extraction on structured docs. *Weakness*: needs good OCR input (it doesn't do handwriting by itself) and was trained mainly on English. Publicly released via HuggingFace.

- **LayoutLMv3 (2022)** – Huang *et al.* (CVPR)

Further unifies text and image masking so both modalities are masked within one model. Shows

SOTA on form (FUNSD) and receipts tasks. *Relevance*: Improved layout model, still needs OCR token input.

- **LayoutXLM (2021)** – Xu *et al.* (arXiv) [21†L3-L11]

A multilingual LayoutLM variant pre-trained on 7 languages (English, Chinese, Japanese, Spanish, French, Italian, German) for form understanding. Introduced the XFUND dataset to test this.

Relevance: Multilingual doc parsing – suggests feasibility of extending to Hindi/Marathi. *Weakness*: Did not include Indic languages, but approach shows multi-script possible.

- **StrucTexT (2021)** – Liu *et al.* (ICCVW)

A unified model for entity labeling/linking in forms via multi-modal Transformer. *Relevance*: Focused on form semantics.

- **Camelot / Table Transformer** – For table detection/extraction (though more engineering tools than papers).

- **DocTR (2021)** – Microsoft/MindEye library implements CNN+Transformer for detection+recognition in one pipeline. Not a paper, but its architecture uses EfficientCNN + CTC. Useful for production.

- **Historical Document Recognition Benchmarks (ICDAR/H-DOC)** – Several papers (e.g. 2017 Arabic, early modern docs) show HTR error rates (CER) in single digits on well-scanned text. On messy field notes, errors jump. *Relevance*: Historical ledger tasks exist but are niche; they highlight that domain-specific HTR is needed (no panacea).

Table Detection & Structure

- **PubTables-1M (2021)** – Smock *et al.* (arXiv) [27†L24-L33]

Massive dataset of 1M tables from scientific PDFs, with fine-grained annotations (cell locations, headers). They propose canonicalization to fix labelling. *Contribution*: Show Transformer detectors achieve SOTA in both table detection and structure parsing. *Relevance*: Great for training a robust table-detector (if farm notes have tables). Drawback: papers/tables domain, different from scribbles.

- **CascadeTabNet (2020)** – Prasad *et al.* (CVPRW) [129†L1-L4]

A CVPR workshop paper with code (MIT license) implementing a Cascade Mask R-CNN + HRNet to detect tables & segment cells in one shot. *Relevance*: Good baseline for table parsing. *Weakness*: CNN-based; outperformed by newer Transformers on benchmarks, but still viable and light.

- **DeepDeSRT (2017)** – Schreiber *et al.* (ICDAR) – classical CNN for table structure. (No code easily reused.)

Key Information Extraction (KIE)

- **Invoice/Receipt Parsers** – Many apply deep learning (LSTMs, Transformers) on OCR output to tag fields. E.g. Microsoft's Invoice Parser, Donut above, or open models like **PaddleOCR-VL** (0.9B VLM) that do page parsing end-to-end [47†L28-L37] . *Relevance*: They extract structured data from invoices/forms. *Weakness*: often require trained templates or OCR.

- **Camelot, Tabula** – Tools for PDF tables via heuristics. (Limited use for arbitrary scribbles.)

Vision-Language Models for Documents

- **Qwen2.5-VL (2025)** – Alibaba (tech report) [36†L28-L36]

A 47B multimodal model fine-tuned for “LLM with vision”. It excels in *document parsing*: “robust structured data extraction from invoices, forms, and tables” [36†L28-L36] . It's top-tier on DocVQA/ KIE. *Relevance*: Shows new generation models can directly parse documents (almost OCR+NLP in one). *Weakness*: Closed architecture (China company), heavy costs.

- **InternVL 3.5 (2024)** – Tsinghua/InternLM team [40†L40-L48]

A family of open Chinese LLMs (~241B) with vision input. Claimed first open model to surpass GPT-4o on multi-modal benchmarks, with innovations (RLHF cascade, dynamic resolution). *Relevance*: Leading open-source contender for vision+LLM tasks, supports multimodal queries. *Weakness*: Chinese focus, heavy.

- **InternVL 2.5 (2023)** – Earlier InternLM version [42†L33-L41]

1.2B VLM achieving 70% on a doc understanding metric (MMMU). Indicates the viability of small-ish multimodal models.

- **MiniCPM-V 4.6 (2024)** – Tsinghua/MMOCR team [44†L149-L158]

A compact 1.3B Chinese VLM optimized for edge. It outperforms larger models on document/OCR tasks like OCRBench and DocVQA [44†L149-L158] . *Relevance*: Shows that you don't need huge models to read text and answer questions. Friendly for offline use.

- **PaddleOCR-VL-0.9B (2025)** – Baidu [47†L28-L37]

A 0.9B VLM (NaViT+ERNIE) trained for documents. Supports 109 languages (including many scripts) and complex elements. It achieves SOTA in page-level & element-level parsing, with blazing inference speed and low resource usage [47†L28-L37] . *Relevance*: Exactly aimed at large-scale doc parsing (including tables, forms) with minimal hardware – ideal for production. *Weakness*: New, but well-engineered by Baidu; open-sourced.

- **Florence-2 (2024)** – Microsoft (CVPR) [38†L21-L29]

A 5.4B promptable vision-language model trained with unified tasks across 126M images. It achieves new SOTA on many vision tasks with prompt-based generation. *Relevance*: Not specialized for docs or OCR, but very strong visual reasoning. Likely good at image QA, but may not extract structured data as JSON easily.

- **GPT-5 / Gemini 2.5 / Claude Opus** – Not published in detail yet. Gemini 2.5 (Google) and Claude Opus (Anthropic) are rumored to be multimodal with doc abilities. Likely extremely capable and very costly (cloud only). They could potentially handle handwriting via few-shot, but unclear support.

- *Comparison*: Among VLMs, **PaddleOCR-VL** is the most practical: small, offline-capable, multi-language and OCR-savvy [47†L28-L37] . The huge ones (Qwen2.5, Intern3.5) have 175B+ parameters and require clouds. MiniCPM and Intern2.5 show small models can work. Product APIs (Google Gemini, Claude) likely excel at reasoning but lock-in/cost is prohibitive.

GitHub Repository Survey

OCR Toolkits

- [PaddleOCR](#) – 70k★ [52†L0-L1] , **Apache-2.0**. Actively maintained (last releases Jul 2026). Python/C++, works offline. Supports **100+ languages**, including Devanagari (Hindi/Marathi) [52†L0-L1] . Includes models for text detection, recognition, and structure (PP-Structure module). Good support for handwritten text (dedicated language models) and messy scans. **Prod-ready**: used widely in industry; GPU helps speed; some complex setup. *Pros*: Very high accuracy, multilingual, includes table/structure parsers. *Cons*: Heavy, many dependencies, relatively steep learning curve.
- [EasyOCR](#) – 29.8k★ [55†L468-L470] , **Apache-2.0**. Python/PyTorch, installable via pip. Supports **80+ languages** (Latin, Chinese, Arabic, Devanagari, etc.) [126†L1-L4] . Offers text detection+recognition, with some handwritten support. Runs fully offline. **Prod-ready**: Fast to use, reasonable accuracy out-of-the-box. *Pros*: Easy API, multi-script, good for prototyping. *Cons*: Less accurate on cursive or very messy text, relatively slow on CPU.
- [DocTR \(Doctr\)](#) – 6.2k★ [127†L1-L4] , **Apache-2.0**. Python/TensorFlow, focuses on document OCR (detection + recognition). Pre-trained models available (English). **Handwriting**: Not specialized for cursive, mainly scene and printed text. Offline support: yes. *Pros*: Good accuracy on clean docs; includes KIE pipelines (e.g. tables). *Cons*: Limited script support, slower, not ideal for cross-lingual.
- [Kraken](#) – 1.0k★ [128†L1-L4] , **Apache-2.0**. Focused on **historical & complex scripts**. Must train models for each script/language. Supports right-to-left, non-Latin. **Handwriting**: yes, it's built for it. **Prod**: Open-source but requires ML expertise to train. *Pros*: Very flexible (OCR, layout analysis). *Cons*: Hard to get high accuracy without large training set; slow.
- *Other OCR*: **Calamari**, **ocropy**, **Tesseract 4+** (LSTM) exist but are less used for messy real-world notes.

Document AI Frameworks

- [Donut](#) – 6.9k★ [65†L1-L4] , MIT license. OCR-free Transformer parser. *Update*: July 2022, active. **Lang**: Unknown (maybe English). **Offline**: Code is open, but GPU needed. *Pros*: Direct JSON outputs; great for structured forms; no OCR dependency [9†L24-L32] . *Cons*: Requires fine-tuning on target doc types; may hallucinate text.
- [MinerU](#) – 75.8k★ [68†L1-L4] , custom “MinerU Open Source License”. *Activity*: Very active (2026). Python, provides CLI/API for multi-format docs (PDF/DOCX/PPTX/XLSX/Images). Internally uses hybrid backends with tiny VLMs. Supports **native parsing of many file types**. **Handwriting**: Yes (claims native multilingual OCR support). *Offline*: Yes (supports CPU-only). *Pros*: Enterprise-grade pipeline (multi-page, API/CLI, multi-thread). *Cons*: Massive codebase, custom license (though Apache-like), steep config.
- **[LayoutLM series (via [HuggingFace](#))** – Pretrained models (LayoutLMv2/v3/XLM). *Stars*: HuggingFace ~600-800★ each. **Apache-2.0**. They require feeding in OCR tokens+bbbox. **Lang**: English (v2/v3), multiple (XLM). *Offline*: Yes, can run locally (GPU recommended). *Pros*: Top-tier on KIE benchmarks, can be fine-tuned on invoices/forms. *Cons*: Depend on external OCR; not HTR-ready.
- [PaddleOCR-Structure \(PP-Structure\)](#) – Part of PaddleOCR (70k★). Specifically for table and form parsing (cell line detection, box merging). *Useful*: For ledger-like tables.
- [GOT-OCR](#) – Google OCR for text orientation (or “Guided OCR with Transformers” – not widely used).
- [DocFormerv2](#) – Research code (MIT) for document understanding with Transformers. Not highly starred but relevant.

- **PaddleOCR-VL** and **Qwen-VL** – These are model releases, not code repos. They have framework support from Paddle or MindStudio, not standard GitHub projects.

Table Understanding Repos

- [CascadeTabNet](#) – 1.5k★ **【129†L1-L4】**, MIT. Code for CVPR'20 table detection+structure. *Languages*: Table images. *Handwriting*: Not specialized. *Offline*: Yes (PyTorch + MMDetection). *Pros*: End-to-end table detection & structure in one model; well-documented. *Cons*: Based on old MMDetection versions; can be finicky to install (requires older PyTorch/CUDA).
- [TabStruct](#) – (no stars listed) – original DeepDeSRT code (OCR often handles tables).
- [Camelot](#) – 16k★. Extract tables from PDFs via rules. Not deep learning, but useful for crisp PDFs. Not for images or handwriting.
- [Excalibur](#) – 2.4k★, GUI+API wrapper around Camelot.
- [TableNet](#) – (less common).

Information Extraction (Invoice/Form Parsers)

- Not many general-purpose repos beyond the above. Most KIE is done with the above models or commercial APIs.
- **InvoiceNet** (older), **PicKnow** (for receipts, focused on products).
- Many wrappers exist (e.g. PaddleOCR's `tools/e2e_inference`, Mindee's `doctr::packages`).

Dataset Survey

Name	Link	License	Size (pages/ images)	Languages	Annotation	Fine-tuning Suitability
IAM (Handwritten)	iamdataset	Non-commercial	~1,500 handwritten pages (150 writers)	English (Latin)	Line-level transcriptions	Standard benchmark for HTR. Useful for base HTR models, but no Hindi/Marathi.
IIIT-HW-Dev (Devanagari)	CVIT-IIIT	Unknown	~5,000 words (new devnagari corpus)	Devanagari (Hindi)	Word-level labels	Good for Hindi HTR fine- tuning (if scripts similar).
Indic Handwritten Word (IIT-HW- Words)	Kaggle IIT-INDIC	Unknown (likely CC)	~10k word images per script (multi- script)	Hindi, Tamil, Bangla, etc.	Word transcriptions	Useful for multi-script pretraining; has Hindi.
Devanagari Handwritten Char	Kaggle Devnagari	Open	46 classes (2000 images each)	Hindi/ Marathi	Character labels	Good for base OCR of Devanagari chars.

Name	Link	License	Size (pages/ images)	Languages	Annotation	Fine-tuning Suitability
FUNSD (Forms)	FUNSD dataset	CC BY-NC-SA 4.0	199 fully annotated form images	English	Word boxes + labeling (elements/ relations)	Great for form fields labelling (English only).
XFUND (Form + Q&A)	XFUND (ACL 2022)	CC BY 4.0	9,000+ form images (8 langs, sim)	7 Indic/East Asian languages + English	Word-level labels (same as FUNSD but 7 translations)	Tests multilingual form parsing; helpful for non-English layout models.
CORD (Receipts)	CORD (ECML 2019)	MIT	~800 receipt images (English, Chinese)	English, Chinese (OCR- heavy)	Word bounding boxes + key- value (table) fields	Good for receipt extraction, not books.
DocVQA (Doc Visual QA)	DocVQA	Free for research	~12k scanned document pages, VQA pairs	Multilingual (mostly English)	Question- answer pairs on doc images	For QA ability; indirectly tests understanding.
RVL-CDIP (Doc Images)	RVL-CDIP (ICDAR)	Non- commercial	400k greyscale pages	English	16-category labels (Invoice, Letter, Form, etc.)	For coarse doc classification, not fine KIE.
DocBank	DocBank (KDD 2020)	CC BY-NC	500k annotated academic paper pages (21 types)	English	Token-level layout tags	Useful for training segmentation of docs (e.g. blocks/lines).

Table

Datasets:

PubTables-1M
[27†L24-L33]
(Open) – 1M
scientific
tables. ICDAR
TableBank (2M
tables). VINCI
(ICDAR2013/19)

Varied
licenses.

Name	Link	License	Size (pages/ images)	Languages	Annotation	Fine-tuning Suitability
Agricultural/ Handbook Records: No large public set. Only small sets like farmers' diaries (not public). <i>Research Gap:</i> True "farm ledger" corpus is missing. Perhaps one must be created for GramIQ.						

Agricultural Accounting Research

No major papers target *farm bookkeeping OCR* specifically. However:

- **AgroExpense-OCR (2026)** – Nuankaew *et al.* (JAIT) [79†L111-L119]
An OCR system for *agricultural expense receipts* using Vision Transformers (FastViT). Tested on a 15k-item dataset, achieving 96.8% accuracy on printed receipts and 92.4% on handwritten ones [79†L111-L119]. *Relevance:* Closest work in "agriculture domain OCR". They also built an app for farmers (scanning receipts, auto-categorizing expenses). Shows that tailored OCR+categorization can work well in practice. *Weakness:* Focused on receipts (not ledger entries) and specific languages (Thai?), not open-sourced data.
- **Farm Blockchain Ledgers (BanQu, Fast Co.)** – Projects using blockchain for farmer transactions (Fast Company) [81†L63-L72]. Not OCR; more about digital finance records.
- *Adjacencies:* Technologies for *mobile farm management apps* exist, but mostly structured data entry or photo-of-crop. No known open dataset of handwritten farm logs. This is a gap GramIQ can fill.

Industrial Document AI Systems

- **Google Document AI** – Cloud APIs for OCR and structured parsing. Offers pre-trained processors (e.g. Invoice parser, Form parser) and a general OCR engine. Uses Google's OCR (Tesseract-based) plus ML for fields. *Pricing:* Variable – e.g. Invoice Parser ~ \$0.10 per 10 pages [84†L128-L137]. *Handwriting:* Basic OCR handles printed; a "Form Parser" can process handwritten forms (e.g. demo with a medical form) [85†L392-L400]. *Features:* Integrates with Google Cloud, provides JSON output for fields. Has human-in-loop tooling (Document AI Workbench for labeling). *Limitations:* Requires

sending docs to cloud; cost accumulates; limited customization unless building a “custom processor”.

- **Azure Document Intelligence (Form Recognizer)** – Microsoft’s IDP API. Has pre-built models (Invoices, ID cards, etc.) and a general **Read** API for OCR (print & handwritten) 【87†L1-L6】. Can be run in cloud or container (Azure Blob container for edge). *Pricing*: ~\$0.50 per 1000 pages for OCR; custom models ~\$1/page. *Handwriting*: The Read API explicitly supports handwritten text extraction 【87†L1-L6】. *Features*: Also provides key-value extraction (via Labeling tool). *Limitations*: Cloud lock-in, licensing for high volumes, needs training for non-prebuilt types.
- **AWS Texttract** – Amazon’s OCR & KIE. Claims to *automatically* extract text, tables, forms from scans 【91†L53-L61】. Supports printed + hand-printed text by trained ML. *Pricing*: e.g. \$0.065 per page for forms extraction (after free tier). *API*: Simple synchronous/batch endpoints. *Limitations*: Results sometimes imperfect on cursive; fields are in JSON but custom queries limited; AWS-bound.
- **Rossum (now Coupa)** – SaaS focused on invoices/AP. Uses proprietary AI (transactional LLM, 276-languages with handwriting support 【94†L104-L112】) and workflows. Offers “AI agents” that parse docs and feed ERP. *Pricing*: Enterprise-tier subscription (quote-based). *Handwriting*: Yes, advertises 276-language support, likely includes major scripts. *Features*: Human-in-loop validation, workflow automation, audit logs. *Limitations*: High cost; narrow (tailored to invoice/PO streams, not general docs).
- **Nanonets** – OCR API service. Provides pre-built models for Invoices, Receipts, Forms; also custom training. Free tier (first 100 invoices) and pay-as-you-go. Claims “no code” training with few examples. *Handwriting*: Has a “Handwritten Forms” product 【95†L9-L12】. *Pricing*: Starts free, then ~\$0.10–0.20 per document. *Limitations*: Closed system, limited control, not open-source.
- **Veryfi** – Specialized OCR for receipts/invoices. Offers on-device SDKs (mobile, edge) or cloud API. Known for privacy (all processing can happen on-device). *Pricing*: Monthly subscription (\$500+ gets some thousands pages) 【100†L1-L4】. *Limitation*: Focused on English/invoice receipt; higher fixed cost.
- **Mindee / docTR** – The company behind DocTR sells API services. Now merged with Rossum to some extent.
- **FormX (formX.ai)** – Unknown, possibly a form-OTP tool. May skip if details are unclear.
- **ABBYY** – Legacy OCR/IDP vendor. *Products*: FlexiCapture, Cloud OCR SDK. Strong handwriting recognition in some languages. *Pricing*: Enterprise licensing (\$\$\$\$) or Cloud SDK (~\$99/month for 5k pages 【98†L1-L4】). *Features*: On-premise option, very mature (layout templates, auto-ML). *Limitation*: Expensive; older tech stack, less ML-centric; integration complexity.
- **UiPath Document Understanding** – RPA platform with IDP. Combines multiple OCR engines (Google, Microsoft, ABBYY, etc.) + AI Fabric. Offers data extraction templates for invoices, receipts. *Pricing*: Enterprise RPA licensing. *Features*: Seamless with automation workflows, human validation station. *Limitations*: Overkill if just standalone OCR; costly; black-box selection of OCR.

Vision-Language Model Comparison

Model	Handwriting & OCR	Reasoning	Multilingual	JSON/ Structure	Long Docs (≥100p)	Cost & Latency
GPT-5 (spec)	Unknown (likely poor without fine-tune)	Very strong (text reasoning)	Multi (prefers English)	Not specialized; needs prompt engineering	Uncertain (likely ~4k token)	Rumored cloud-only, very high cost
Gemini 2.5 Pro	Strong on OCR/ diagrams (paper leak videos); likely good English handwriting	Top-tier LLM reasoning	Multi (particularly English/ Chinese)	Possibly via Vision API or descriptions	40k tokens (Llama size?)	Google Cloud, expensive per image
Claude Opus 3/4	Strong multimodal (reports chemical graphs)	Excellent (safety-tuned)	English-focused	No direct JSON API; need system prompts	Possibly large (~90k context)	Cloud API, unknown pricing, likely high latency
Qwen2.5-VL 【36†L28-L36】	Excellent (specifically mentions invoices/ forms)	Good (big model)	Chinese + some English, supports 276 langs?	Direct structured output (it's "vision-language") 【36†L28-L36】	Likely ~4k tokens	Closed Chinese cloud; high GPU requirements
InternVL (3.5) 【40†L40-L48】	Very strong (competition with GPT-4o); likely good OCR too	Strong (fine-tuned with RLHF)	Chinese primary; open-source allows few-shot in other langs?	Via prompting (no built-in JSON)	~32k tokens	Open-source (can self-host), costs = own infra; inference intensive
MiniCPM-V 4.6 【44†L149-L158】	Good at reading text (beats larger models on OCRBench)	Reasoning (edge case?)	Chinese (maybe bilingual?)	Q&A style (DocVQA: 68/80 correct) 【44†L149-L158】	~2k tokens (edge device)	Fast (1.3B model); edge-friendly (GPU ~4GB)

Model	Handwriting & OCR	Reasoning	Multilingual	JSON/ Structure	Long Docs (≥100p)	Cost & Latency
PaddleOCR-VL 0.9B 【47†L28-L37】	Excellent (designed for OCR and tables)	Moderate (not an LLM, but can parse structure)	109 languages support 【47†L28-L37】	Yes – outputs document JSON fields SOTA 【47†L28-L37】	Handles pages (as image); multi-page via batching	Very efficient (0.9B), low latency, can run on moderate GPU, open-source
Florence-2 【38†L21-L29】	Decent (trained on scene text too)	Very strong visual reasoning	Weak on non-English text (focus on images)	Not focused on structured extraction (image captions mostly)	~4k tokens	Hosted via Azure; very expensive (foundation model)
Claude 3 Opus (Anthropic)	Good (multimodal), unknown on cursive	Very good (safety, chain-of-thought)	English primarily	Need prompt to JSON	~30k tokens	Cloud only (Claude Cloud), likely high

Model	Handwriting & OCR	Reasoning	Multilingual	JSON/ Structure	Long Docs ($\geq 100p$)	Cost & Latency
<i>Benchmark notes:</i> PaddleOCR-VL's creators report outperforming larger models on document parsing tasks [47†L28-L37] . MiniCPM-V achieved 68/80 on DocVQA [44†L149-L158] , indicating reasonable understanding. Open models lag behind closed giants on pure reasoning, but excel in cost/latency (MiniCPM <1s per query on GPU). Large models (Gemini, Claude) likely zero-shot handle OCR queries, but cost ~\$1+ per API call.						

Recommended System Architecture

Workflow: Farmer photo → Preprocessing → Layout/Line Segmentation → OCR/HTR → Structure Parsing → Transaction Segmentation → Field Extraction → Category Classification → Confidence Scoring → Human Review → Database [116†] .

[116†embed_image] *Figure: Example of a messy farm ledger page. Our pipeline will handle such pages end-to-end.*

- 1. Image Enhancement:** Clean up the photo: deskew, adjust contrast, remove noise. This boosts OCR on messy notebooks. Quick techniques (CLAHE, bilateral filters) or trainable enhancers can be applied.
- 2. Layout Detection:** Use a model like **CascadeTabNet** or **YOLO** tuned to find lines/tables and isolate “columns” or entry blocks. Farm ledgers may have drawn lines or free-form. A CNN detector can locate bounding boxes of transaction rows or date columns.
- 3. Handwritten OCR:** For each detected text region, run a specialized HTR model. We might use **TrOCR** or **PaddleOCR**'s HTR models, or even **MiniCPM-V** in OCR mode (since it reads text well [44†L149-

L158]). If data permits, fine-tune on similar Hindi/Marathi handwriting (perhaps using collected samples).

4. **Document Understanding:** Combine OCR outputs and image context in a DocAI model. Options:
 5. Use **Donut** or **PaddleOCR-VL** to parse the page image into structured JSON (field:value pairs). For example, train Donut on synthetic farm ledger layouts so it outputs JSON like `{"Date": "...", "Item": "...", "Amount": ...}`.
 6. Or use a *text-and-layout* model like **LayoutLMv3** to label tokens and then apply rules. The goal is to identify each transaction entry (date, description, qty, amount, vendor, etc.) as a record.
7. **Transaction Segmentation:** The document understanding step should segment rows. If not, we can post-process OCR lines by detecting date patterns or line breaks. For example, split on new-dates or bullet-lines.
8. **Field Extraction:** For each entry, extract finer details: crop name, category (seeds, fertilizer, etc.), payment type. This can be done via regex/NLP on description, or using an LLM prompt: "Given this entry, classify it." We may fine-tune a classifier (or use internal taxonomy) to tag categories and extra fields (units, vendor).
9. **Category Classification:** Map keywords to farm expense/income categories. A simple lookup (e.g., Hindi word "बीज" means "Seeds") or a small machine learning classifier. We can also use a language model prompt (chain-of-thought) if uncertain.
10. **Confidence Scoring:** Compute confidence at each step (OCR confidence, model probability). If any part is below threshold (e.g. illegible handwriting), flag for review. For example, if OCR returns <80% certainty, mark that transaction for human check.
11. **Human Review Interface:** Provide a UI for an agronomist to quickly verify low-confidence entries (show the image snippet and parsed text side-by-side). This ensures accuracy (because in farming, a rupee off can hurt).
12. **Database Ingestion:** Validated JSON records are automatically stored in GramIQ's DB (we have the schema ready). Each JSON field maps to columns (date, vendor, category, amounts, notes).

Why these components? Image cleanup is standard in OCR pipelines to boost accuracy. Layout detection isolates messy free-form text (ledger pages are not uniform forms). Using a dedicated HTR step ensures we don't rely solely on generic OCR (which is weak on cursive scripts). The doc-understanding model (Donut/PaddleOCR-VL) unifies text+layout analysis and outputs data directly. Breaking each page into transactions and fields handles the ledger semantics. Finally, a human loop catches the inevitable errors, making the system practical.

Risks and Mitigations

- **Messy Handwriting & Languages:** If text is illegible or local slang is used, OCR will fail. *Mitigation:* Collect representative samples early; augment training data (back-translate Marathi→Hindi→English, etc.).
- **Layout Variability:** Without fixed columns, segmentation may misalign. *Mitigation:* Use flexible detectors (object detection on date patterns) and robust image analysis (skew correction).
- **Mixed Scripts:** Some entries mix Hindi/English (or Marathi-English). A single model may struggle. *Mitigation:* Use a Unicode OCR engine (PaddleOCR, EasyOCR) that can output both scripts.

- **Computational Load:** Large VLMs (Gemini, Qwen) might be tempting but costly. *Mitigation:* Favor lightweight models (MiniCPM-V, PaddleOCR-VL) that run on local GPU/CPU.
- **Data Privacy:** Farmer's books are personal. *Mitigation:* Ensure processing can be done on-device or secure cloud; anonymize if needed.
- **Errors in Categorization:** Automatic labelling may misclassify an expense. *Mitigation:* Provide manual override in UI; use domain-specific taxonomies to improve.

Future Research Directions

- **Farm Ledger Datasets:** Create a publicly-shared dataset of annotated farmer notebooks (Hindi/Marathi/English). This would be *world's first farm ledger corpus*.
- **Multilingual HTR Models:** Fine-tune or pre-train Transformer OCR on Indic scripts, including transliteration. Possibly use LLaMA/LLM to hallucinate/rescore uncertain readings.
- **End-to-End FarmDoc Model:** Inspired by Donut, develop "FarmDonut": a transformer pre-trained on agriculture forms and receipts, able to output JSON fields for farm expenses.
- **Vision-LLM for Farming:** Experiment with models like MiniCPM-V or InternVL by prompting "Extract transactions from this agricultural ledger page." Evaluate whether a general VLM can replace parts of the pipeline.
- **Precision Agriculture Taxonomy:** Enhance field extraction by linking items to databases (seed varieties, fertilizer types) using KGs. For example, prompt an LLM: "The entry 'Rubber Seeds 20kg' – is this seeds or produce?" to improve accuracy.

Implementation Roadmap

1. **Prototype OCR & Parsing:** Collect 50 sample pages. Test existing OCR (Paddle, EasyOCR) and Donut/PaddleOCR-VL on these. Evaluate CER/WER.
2. **Develop Layout Detector:** Label lines/entries in a handful of pages, train a small object detector (e.g. YOLOv8) to find date/entry boxes.
3. **Build Integration Pipeline:** Glue preprocessing → layout → OCR → JSON parser (Donut). Output JSON schema.
4. **Category Classifier:** List farm categories (seeds, fertilizer, labor, etc.). Use keywords to define each. Train a small multi-class model on sample entries.
5. **UI for Review:** Design a simple web interface (can be internal) showing image snippets and parsed text, allowing corrections.
6. **Feedback Loop:** As humans correct data, use that to fine-tune models (active learning).
7. **Scale Up:** Deploy on cloud or on-prem GPU. Optimize model sizes (distill if needed).
8. **Monitoring:** Build logs for OCR confidence trends; adjust models if error rates rise.

Technology Stack Recommendation

- **Programming:** Python for prototyping (PyTorch/TensorFlow for models). REST API (FastAPI/Flask) for inference service.
- **OCR/HTR:** PaddleOCR toolkit + custom trained recognizer for Devanagari.
- **Document Parsing:** Donut or **PaddleOCR-VL** (0.9B) via [PaddleOCR-VL repo](#) (Baidu). Use GPU inference.
- **Layout Detection:** YOLOv8 (Ultralytics) or CascadeTabNet (if tables).

- **Classifier:** Lightweight Transformer (DistilBERT multilingual) or prompt-engineered LLM (e.g. Llama2-chat) for categories.
- **UI & Database:** React frontend (for human review), Node.js backend. Database: PostgreSQL for structured records.
- **Cloud/Infra:** AWS/GCP VM with 1 GPU (A100/V100) for model serving. Containerize with Docker+Kubernetes for scale.
- **Monitoring:** Use MLflow or similar for model versioning, plus custom scripts for accuracy metrics on a holdout set.

This tech stack blends bleeding-edge AI with practical engineering tools, aligning with GramIQ's needs. Throughout, we focus on proven solutions (PaddleOCR, LayoutLM/Donut, UIPath-like workflows) but stay skeptical – we'll iterate on the real data, not just papers. Together, these steps will deliver a production-grade system to digitize farm notebooks into actionable financial data, complete with citations and proven benchmarks to boot.

Sources: We referenced state-of-the-art models and benchmarks [7†L19-L27] [9†L24-L32] [12†L25-L33] [79†L111-L119] [84†L128-L137] [87†L1-L6] [91†L53-L61] [94†L104-L112] [98†L1-L3] , plus various GitHub repos (PaddleOCR [52†L0-L1] [122†L1-L4] , EasyOCR [55†L468-L470] [126†L1-L4] , DocTR [127†L1-L4] , Kraken [128†L1-L4] , CascadeTabNet [129†L1-L4] , etc.) for concrete data on stars, license, and capabilities. These inform the recommendations and ensure our report is grounded in the latest research and industry practice.
